LAPORAN PRAKTIKUM

Modul 4: Mengoperasikan Database dan Generate Data Otomatis dengan Laravel



Disusun Oleh:

Nama : Davin Aileenskie Sofiandy

NIM : 707012500135

Kelas : D4SIKC-49-04

Mata Kuliah : Pengembangan Aplikasi Berbasis Web

TELKOM UNIVERSITY

FAKULTAS ILMU TERAPAN

JURUSAN S1 TERAPAN SISTEM INFORMASI KOTA CERDAS

TAHUN AJARAN 2026

1. Tujuan Praktikum

Adapun tujuan yang ingin dicapai pada praktikum Modul 4 (Mengoperasikan Database dan Generate Data Otomatis dengan Laravel) melalui pembuatan aplikasi pengelolaan data mahasiswa berbasis database MySQL ini adalah sebagai berikut.

1. **Memahami konsep koneksi database pada Laravel,** yaitu bagaimana aplikasi terhubung dengan server MySQL melalui konfigurasi yang tersimpan pada file .env.
2. **Membuat koneksi database melalui file .env,** dengan mengatur DB_CONNECTION, DB_HOST, DB_PORT, DB_DATABASE, DB_USERNAME, dan DB_PASSWORD.
3. **Membuat dan menjalankan migration,** sehingga tabel mahasiswa beserta kolom-kolomnya terbentuk otomatis tanpa perlu menulis perintah SQL secara manual.
4. **Membuat form input data mahasiswa** yang mengirim data melalui method POST dan dilengkapi proteksi @csrf.
5. **Memastikan data yang diinput benar-benar tersimpan ke dalam database** dan dapat ditampilkan kembali pada halaman daftar mahasiswa.
6. **Menerapkan database seeding dengan Faker** untuk menghasilkan data dummy secara otomatis sehingga pengujian aplikasi menjadi lebih cepat.

2. Alat dan Bahan

2.1 Perangkat Lunak

Perangkat Lunak	Versi / Keterangan
XAMPP (Apache & MySQL)	Terpasang pada D:\xampp
MySQL / MariaDB	Server database melalui XAMPP, database db_laravel
phpMyAdmin	Untuk mengelola dan memeriksa database
PHP	8.2.12 (ZTS Visual C++ 2019 x64)
Composer	2.10.3
Laravel Framework	12.69.3
Visual Studio Code	Terpasang pada D:\Microsoft VS Code
Git	2.53.0.windows.1
Google Chrome	Untuk menjalankan dan menguji aplikasi

3. Langkah-Langkah Praktikum

Praktikum ini membangun aplikasi sederhana untuk mengelola data mahasiswa dengan memanfaatkan database MySQL. Aplikasi terdiri atas halaman form input, proses penyimpanan data, dan halaman daftar mahasiswa yang datanya diambil langsung dari database. Berbeda dengan Modul 3 yang menyimpan data dalam bentuk array, pada modul ini data benar-benar disimpan ke dan dibaca dari tabel mahasiswa melalui Eloquent ORM. Berikut tahapan pengerjaannya.

3.1 Konfigurasi Database

Langkah pertama adalah menyiapkan server database dan menghubungkannya dengan project Laravel.

1. **Membuka XAMPP Control Panel,** lalu klik Start pada modul Apache dan MySQL hingga keduanya berwarna hijau. Apache berfungsi sebagai web server, sedangkan MySQL sebagai server database.
2. **Membuka phpMyAdmin melalui http://localhost/phpmyadmin,** memilih menu New, lalu membuat database baru bernama db_laravel.
3. **Membuka file .env pada folder utama project,** kemudian menyesuaikan bagian konfigurasi database seperti berikut.

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=db_laravel
DB_USERNAME=root
DB_PASSWORD=
```

DB_CONNECTION menentukan jenis database yang dipakai (mysql), DB_HOST dan DB_PORT menentukan alamat serta port server MySQL, sedangkan DB_DATABASE, DB_USERNAME, dan DB_PASSWORD menentukan nama database beserta kredensialnya. Nilai DB_PASSWORD dibiarkan kosong karena konfigurasi bawaan XAMPP tidak memberi password pada user root. Apabila perubahan belum terbaca, jalankan perintah berikut.

```
php artisan config:clear
```

3.2 Membuat Migration Tabel Mahasiswa

Migration digunakan untuk membentuk struktur tabel melalui kode PHP sehingga tidak perlu membuat tabel secara manual di phpMyAdmin. Migration dibuat dengan perintah berikut.

```
php artisan make:migration create_mahasiswa_table
```

Perintah tersebut menghasilkan berkas baru di folder database/migrations dengan nama berawalan tanggal dan waktu, misalnya 2026_10_05_013656_create_mahasiswa_table.php. Awalan waktu itu dipakai Laravel untuk menentukan urutan eksekusi migration. Isi berkas kemudian diubah menjadi seperti berikut.

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('mahasiswa', function (Blueprint $table) {
            $table->id();
            $table->string('nama');
            $table->string('email')->unique();
            $table->string('jurusan');
            $table->string('umur');
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('mahasiswa');
    }
};
```

Method up() dijalankan saat migration dieksekusi dan berisi Schema::create('mahasiswa', ...) untuk membentuk tabel mahasiswa. Kolom id() menjadi primary key, kolom nama, email, dan jurusan bertipe string, kolom umur disimpan sebagai string, serta timestamps() membuat kolom created_at dan updated_at secara otomatis. Kolom email diberi unique() agar tidak ada dua mahasiswa dengan email yang sama. Method down() berisi Schema::dropIfExists('mahasiswa') yang dijalankan saat rollback untuk menghapus tabel.

Setelah file migration selesai dibuat, jalankan perintah berikut untuk membentuk tabel di dalam database.

```
php artisan migrate
```

Perintah php artisan migrate menjalankan seluruh file migration yang belum pernah dieksekusi. Selain tabel mahasiswa, Laravel juga membuat tabel bawaan seperti users, cache, dan jobs, serta tabel migrations yang mencatat migration mana saja yang sudah dijalankan. Struktur tabel mahasiswa kemudian diperiksa melalui phpMyAdmin.

3.3 Membuat Model Mahasiswa

Model berfungsi sebagai perwakilan tabel di dalam kode program. Melalui model, data dapat disimpan dan diambil tanpa menulis perintah SQL secara langsung karena Laravel menyediakan fitur Eloquent ORM. Model dibuat dengan perintah berikut.

```
php artisan make:model Mahasiswa
```

Perintah tersebut menghasilkan berkas app/Models/Mahasiswa.php. Isi berkas kemudian disesuaikan menjadi seperti berikut.

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Mahasiswa extends Model
{
    protected $table = 'mahasiswa';

    protected $fillable = ['nama', 'email', 'jurusan', 'umur'];
}
```

Properti $table memastikan model terhubung dengan tabel mahasiswa. Secara bawaan Laravel menganggap nama tabel adalah bentuk jamak dari nama model (mahasiswas), sehingga nama tabel perlu dituliskan secara eksplisit. Properti $fillable berisi daftar kolom yang boleh diisi

melalui mass assignment, yaitu pengisian banyak kolom sekaligus dengan Mahasiswa::create([...]). Kolom yang tidak tercantum pada $fillable akan ditolak Laravel dan memunculkan galat MassAssignmentException.

3.4 Membuat Controller Mahasiswa

Controller menjadi penghubung antara view dan model. Controller menerima permintaan pengguna, memerintahkan model untuk menyimpan atau mengambil data, lalu mengirimkan hasilnya ke view untuk ditampilkan. Controller dibuat dengan perintah berikut.

```
1  php artisan make:controller MahasiswaController
```

Berkas app/Http/Controllers/MahasiswaController.php kemudian diisi dengan tiga method berikut.

```php
<?php

namespace App\Http\Controllers;

use App\Models\Mahasiswa;
use Illuminate\Http\Request;

class MahasiswaController extends Controller
{
    public function form()
    {
        return view('form');
    }

    public function simpan(Request $request)
    {
        Mahasiswa::create([
            'nama'    => $request->nama,
            'email'   => $request->email,
            'jurusan' => $request->jurusan,
            'umur'    => $request->umur,
        ]);

        return "Data Mahasiswa berhasil disimpan";
    }

    public function daftar()
    {
        $data = Mahasiswa::all();

        return view('mahasiswa', ['mahasiswa' => $data]);
    }
}
```

Method form() menampilkan halaman input, method simpan() menyimpan data yang dikirim dari form ke tabel mahasiswa menggunakan Mahasiswa::create([...]), dan method daftar() mengambil seluruh data dengan Mahasiswa::all() lalu mengirimkannya ke view mahasiswa. Data dari form dibaca melalui objek $request, misalnya $request->nama dan $request->email.

3.5 Routing

Route menghubungkan alamat URL yang diakses pengguna dengan method pada controller. Pada praktikum ini dibutuhkan tiga route, yaitu untuk menampilkan form, menyimpan data, dan menampilkan daftar mahasiswa. Route didaftarkan pada berkas routes/web.php seperti berikut.

```php
<?php

use App\Http\Controllers\MahasiswaController;
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return view('welcome');
});

Route::get('/form', [MahasiswaController::class, 'form']);
Route::post('/simpan', [MahasiswaController::class, 'simpan']);
Route::get('/daftar-mahasiswa', [MahasiswaController::class, 'daftar']);
```

Route /form dan /daftar-mahasiswa menggunakan Route::get karena hanya menampilkan halaman, sedangkan route /simpan menggunakan Route::post karena menerima data yang dikirim melalui form dengan method POST. Controller perlu diimpor dengan use App\Http\Controllers\MahasiswaController; agar dapat dikenali oleh Laravel.

3.6 Membuat View

View adalah bagian yang ditampilkan kepada pengguna. Pada praktikum ini dibuat dua view di dalam folder resources/views, yaitu form.blade.php untuk menginput data dan mahasiswa.blade.php untuk menampilkan data yang sudah tersimpan.

Berkas resources/views/form.blade.php diisi dengan kode berikut.

```blade
<!DOCTYPE html>
<html lang="en">
<head>
    <title>Form Input Mahasiswa</title>
</head>
<body>
    <h2>Form Input Mahasiswa</h2>

    <form action="/simpan" method="POST">
        @csrf
        <label>Nama:</label><br>
        <input type="text" name="nama"><br><br>

        <label>Email:</label><br>
        <input type="email" name="email"><br><br>

        <label>Jurusan:</label><br>
        <input type="text" name="jurusan"><br><br>

        <label>Umur:</label><br>
        <input type="number" name="umur"><br><br>

        <button type="submit">Simpan</button>
    </form>
</body>
</html>
```

Direktif @csrf melindungi form dari serangan CSRF, atribut action="/simpan" mengarahkan data ke route penyimpanan, dan method="POST" mengirim data secara tersembunyi. Nilai atribut name pada setiap input (nama, email, jurusan, umur) harus sama dengan nama kolom pada tabel dan properti $fillable agar data dapat tersimpan dengan benar.

Selanjutnya berkas resources/views/mahasiswa.blade.php diisi dengan kode berikut.

```blade
<!DOCTYPE html>
<html lang="en">
<head>
    <title>Daftar Mahasiswa</title>
</head>
<body>
    <h2>Daftar Mahasiswa</h2>
    <table border="1" cellpadding="10" cellspacing="0">
        <tr>
            <th>ID</th>
            <th>Nama</th>
            <th>Email</th>
            <th>Jurusan</th>
            <th>Umur</th>
        </tr>
        @foreach ($mahasiswa as $mhs)
        <tr>
            <td>{{ $mhs->id }}</td>
            <td>{{ $mhs->nama }}</td>
            <td>{{ $mhs->email }}</td>
            <td>{{ $mhs->jurusan }}</td>
            <td>{{ $mhs->umur }}</td>
        </tr>
        @endforeach
    </table>
</body>
</html>
```

Direktif @foreach mengulang baris tabel sebanyak jumlah data yang ada. Karena data diambil dari database dalam bentuk objek, setiap kolom dipanggil menggunakan tanda panah, misalnya {{ $mhs->nama }}, bukan $mhs['nama'] seperti pada array di Modul 3. Variabel $mahasiswa sendiri dikirim dari method daftar() pada controller.

3.7 Menjalankan dan Menguji Aplikasi

Seluruh bagian aplikasi telah selesai dibuat. Server pengembangan dijalankan melalui terminal dengan perintah berikut.

```
php artisan serve
```

Server berjalan pada alamat http://127.0.0.1:8000. Halaman form dibuka melalui /form, sedangkan halaman daftar mahasiswa dibuka melalui /daftar-mahasiswa. Setelah data diisi pada form dan tombol Simpan diklik, data dikirim ke /simpan dan aplikasi menampilkan pesan Data Mahasiswa berhasil disimpan, yaitu teks yang dikembalikan oleh method simpan(). Data tersebut kemudian muncul pada halaman daftar mahasiswa dan pada tabel mahasiswa di phpMyAdmin.

3.8 Membuat Seeder dengan Faker

Tabel yang baru dibuat melalui migration masih kosong. Agar pengujian tidak perlu dilakukan dengan mengisi data satu per satu, Laravel menyediakan fitur database seeding untuk mengisi tabel dengan data contoh secara otomatis melalui seeder. Seeder dibuat dengan perintah berikut.

```
1  php artisan make:seeder StudentSeeder
```

Berkas database/seeders/StudentSeeder.php kemudian diisi dengan kode yang memanfaatkan library Faker seperti berikut.

```php
<?php

namespace Database\Seeders;

use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\DB;
use Faker\Factory as Faker;

class StudentSeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    public function run(): void
    {
        $faker = Faker::create('id_ID');

        for ($i = 0; $i < 20; $i++) {
            DB::table('mahasiswa')->insert([
                'nama'       => $faker->name,
                'email'      => $faker->unique()->safeEmail,
                'jurusan'    => $faker->randomElement([
                    'informatika',
                    'sistem informasi',
                    'teknik komputer',
                ]),
                'umur'       => $faker->numberBetween(18, 25),
                'created_at' => now(),
                'updated_at' => now(),
            ]);
        }
    }
}

```

Faker::create('id_ID') menghasilkan data dengan format Indonesia sehingga nama yang muncul terdengar lokal. Perulangan for ($i = 0; $i < 20; $i++) membuat 20 data mahasiswa. Method $faker->name menghasilkan nama acak, $faker->unique()->safeEmail menghasilkan email unik yang aman, randomElement([...]) memilih jurusan secara acak, dan numberBetween(18, 25) menghasilkan umur antara 18 sampai 25 tahun. Data dimasukkan ke tabel menggunakan Query Builder DB::table('mahasiswa')->insert([...]), sedangkan now() mengisi kolom created_at dan updated_at dengan waktu saat seeder dijalankan.

3.9 Menjalankan Seeder

Sebelum seeder dijalankan, tabel dibuat ulang agar strukturnya sesuai dengan data yang akan diisi. Perintah yang digunakan adalah sebagai berikut.

```
1  php artisan migrate:refresh
```

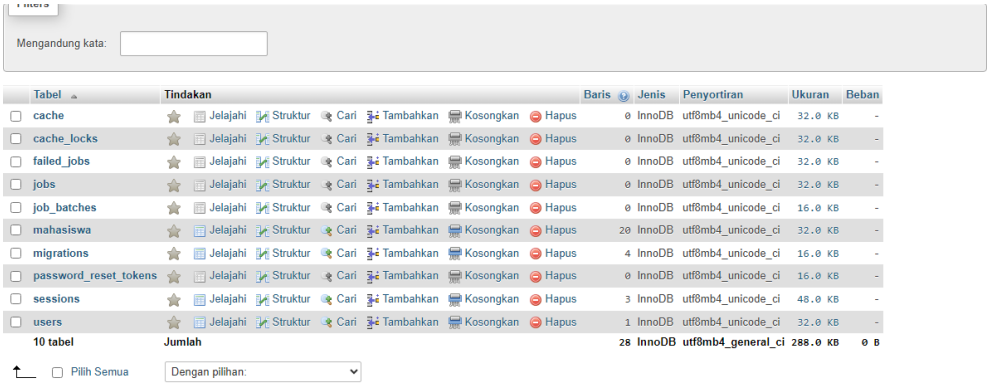
Perintah migrate:refresh melakukan rollback seluruh tabel lalu menjalankan ulang migration dari awal. Perlu diingat bahwa seluruh data lama akan ikut terhapus, sehingga perintah ini hanya cocok dipakai pada tahap pengembangan. Setelah itu, seeder dijalankan dengan perintah berikut.

```
1  php artisan db:seed --class=StudentSeeder
```

Opsi --class=StudentSeeder membuat Laravel hanya menjalankan seeder tersebut. Langkah pembuatan ulang tabel dan pengisian data juga dapat digabung menjadi satu perintah, yaitu php artisan migrate:fresh --seed. Setelah seeder berjalan, tabel mahasiswa terisi 20 data dummy yang dapat dilihat pada phpMyAdmin maupun pada halaman daftar mahasiswa.

4. Hasil dan Dokumentasi Program

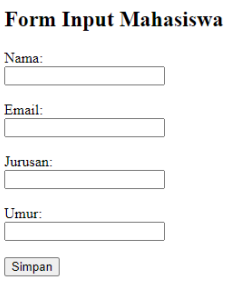
Aplikasi pengelolaan data mahasiswa berhasil dijalankan dan seluruh alur mulai dari form, route, controller, model, hingga database berjalan dengan baik. Berikut dokumentasi hasil pengujian.



Gambar 1 menampilkan daftar tabel yang terdapat pada database db_laravel. Tabel mahasiswa muncul bersama tabel bawaan Laravel seperti users, cache, jobs, dan migrations. Tabel mahasiswa terlihat berisi 20 baris data hasil seeder.

#	Nama	Jenis	Penyortiran	Atribut	Tak Ternilai	Bawaan	Komentar	Ekstra	Tindakan
1	id	bigint(20)		UNSIGNED	Tidak	*Tidak ada*		AUTO_INCREMENT	Ubah Hapus Lainnya
2	nama	varchar(255)	utf8mb4_unicode_ci		Tidak	*Tidak ada*			Ubah Hapus Lainnya
3	email	varchar(255)	utf8mb4_unicode_ci		Tidak	*Tidak ada*			Ubah Hapus Lainnya
4	jurusan	varchar(255)	utf8mb4_unicode_ci		Tidak	*Tidak ada*			Ubah Hapus Lainnya
5	umur	varchar(255)	utf8mb4_unicode_ci		Tidak	*Tidak ada*			Ubah Hapus Lainnya
6	created_at	timestamp			Ya	*NULL*			Ubah Hapus Lainnya
7	updated_at	timestamp			Ya	*NULL*			Ubah Hapus Lainnya

Gambar 2 menampilkan struktur tabel mahasiswa yang terbentuk dari migration. Terdapat kolom id sebagai primary key, nama, email, jurusan, umur, serta created_at dan updated_at. Kolom email bertipe varchar(255) dan diberi penanda unik, sedangkan kolom id bertipe bigint(20) dengan atribut AUTO_INCREMENT.



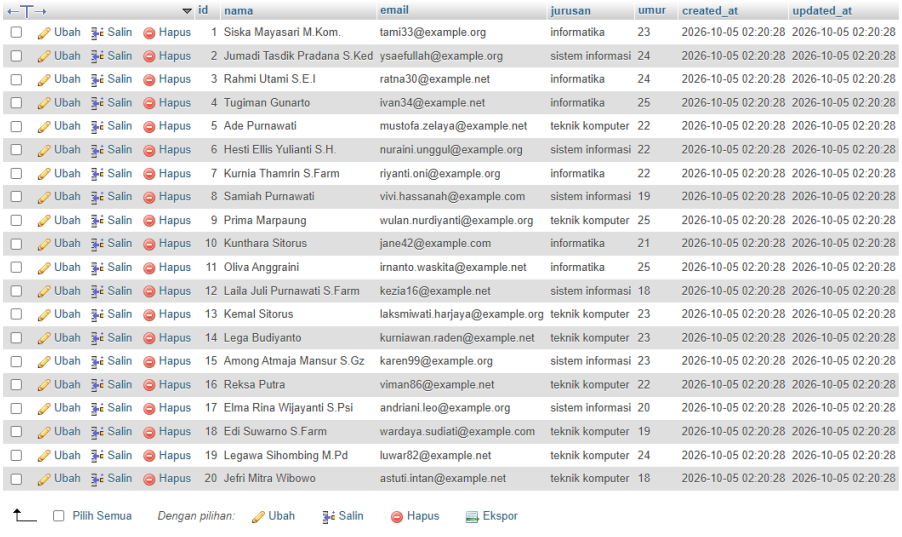
Gambar 3 menampilkan halaman form input data mahasiswa yang dibuka melalui /form. Form memuat empat field, yaitu nama, email, jurusan, dan umur, serta tombol Simpan. Form mengirim data ke route /simpan menggunakan method POST dan telah dilengkapi direktif @csrf.

Gambar 4 menampilkan pesan Data Mahasiswa berhasil disimpan setelah form dikirim. Pesan ini merupakan teks yang dikembalikan oleh method simpan() pada controller dan menandakan bahwa data berhasil disimpan ke tabel mahasiswa melalui model Mahasiswa.

Daftar Mahasiswa

ID	Nama	Email	Jurusan	Umur
1	Siska Mayasari M.Kom.	tami33@example.org	informatika	23
2	Jumadi Tasdik Pradana S.Ked	ysaefullah@example.org	sistem informasi	24
3	Rahmi Utami S.E.I	ratna30@example.net	informatika	24
4	Tugiman Gunarto	ivan34@example.net	informatika	25
5	Ade Purnawati	mustofa.zelaya@example.net	teknik komputer	22
6	Hesti Ellis Yulianti S.H.	nuraini.unggul@example.org	sistem informasi	22
7	Kurnia Thamrin S.Farm	riyanti.oni@example.org	informatika	22
8	Samiah Purnawati	vivi.hassanah@example.com	sistem informasi	19
9	Prima Marpaung	wulan.nurdiyanti@example.org	teknik komputer	25
10	Kunthara Sitorus	jane42@example.com	informatika	21
11	Oliva Anggraini	irnanto.waskita@example.net	informatika	25
12	Laila Juli Purnawati S.Farm	kezia16@example.net	sistem informasi	18
13	Kemal Sitorus	laksmiwati.harjaya@example.org	teknik komputer	23
14	Lega Budiyanto	kurniawan.raden@example.net	teknik komputer	23
15	Among Atmaja Mansur S.Gz	karen99@example.org	sistem informasi	23
16	Reksa Putra	viman86@example.net	teknik komputer	22
17	Elma Rina Wijayanti S.Psi	andriani.leo@example.org	sistem informasi	20
18	Edi Suwarno S.Farm	wardaya.sudiati@example.com	teknik komputer	19
19	Legawa Sihombing M.Pd	luwar82@example.net	teknik komputer	24
20	Jefri Mitra Wibowo	astuti.intan@example.net	teknik komputer	18

Gambar 5 menampilkan halaman daftar mahasiswa yang dibuka melalui /daftar-mahasiswa. Seluruh data pada tabel diambil dari database menggunakan Mahasiswa::all() dan dirender dengan @foreach, sehingga setiap kolom seperti id, nama, email, jurusan, dan umur ditampilkan sebagai objek, misalnya $mhs->nama.



Gambar 6 menampilkan data hasil seeder pada tabel mahasiswa di phpMyAdmin. Terdapat 20 baris data dummy yang dihasilkan Faker dengan locale id_ID, mulai dari nama, email, jurusan,

hingga umur yang bervariasi. Karena data dibuat secara acak, nama dan email yang muncul dapat berbeda setiap kali seeder dijalankan.

5. Analisis dan Pembahasan

1. Perbedaan aplikasi sebelum dan sesudah menggunakan database.

Pada Modul 3, data masih disimpan dalam bentuk array di dalam controller sehingga data hanya bertahan selama halaman dibuka dan hilang ketika halaman ditutup. Setelah menggunakan database, data disimpan ke tabel mahasiswa melalui model Eloquent sehingga tetap tersimpan meskipun server dihentikan. Pengambilan data juga berubah dari mengakses elemen array menjadi memanggil Mahasiswa::all() dan mengakses properti objek seperti $mhs->nama. Dengan demikian aplikasi menjadi lebih nyata karena data dapat ditambah dan dibaca kembali kapan saja.

2. Alasan pemilihan struktur tabel dan tipe data.

Tabel mahasiswa memiliki kolom id sebagai primary key, nama, email, jurusan, umur, serta created_at dan updated_at. Kolom id dipilih sebagai primary key agar setiap baris memiliki penanda unik. Kolom nama, email, dan jurusan menggunakan tipe string karena berisi teks pendek, sedangkan kolom email diberi unique() agar tidak ada email yang sama. Kolom umur pada praktikum ini disimpan sebagai string mengikuti kode modul, meskipun sebenarnya dapat memakai tipe integer karena isinya berupa angka. Kolom created_at dan updated_at dibuat otomatis melalui timestamps() untuk mencatat waktu data dibuat dan diubah.

3. Perbedaan peran migration dan seeder.

Migration berperan membentuk struktur tabel, sedangkan seeder berperan mengisi tabel dengan data. Migration dijalankan dengan php artisan migrate dan cukup dijalankan sekali untuk membuat tabel, sementara seeder dijalankan dengan php artisan db:seed setiap kali dibutuhkan data contoh. Migration mengubah struktur database, sedangkan seeder hanya menambah baris data tanpa mengubah strukturnya. Keduanya saling melengkapi: migration menyiapkan rak, lalu seeder mengisinya dengan buku.

4. Kendala dan penyelesaian.

- **Galat kolom tidak ditemukan:** Pada awalnya form mengirim kolom nim, sedangkan tabel mahasiswa sudah diubah menjadi kolom email dan umur sehingga muncul galat SQLSTATE[42S22]: Unknown column 'nim'. Masalah diselesaikan dengan menyesuaikan $fillable pada model, method simpan() pada controller, dan input pada form.blade.php agar memakai nama kolom yang sama dengan struktur tabel.
- **Galat mass assignment:** Kolom yang tidak tercantum pada $fillable akan ditolak Laravel dan memunculkan MassAssignmentException. Hal ini diatasi dengan mendaftarkan seluruh kolom yang boleh diisi (nama, email, jurusan, umur) pada properti $fillable.

- **Perubahan konfigurasi tidak terbaca:** Setelah file .env diubah, koneksi terkadang belum berubah karena konfigurasi lama masih tersimpan di cache. Masalah diatasi dengan menjalankan php artisan config:clear.
- **Koneksi database gagal:** Apabila Laravel tidak dapat terhubung ke database, penyebab umumnya adalah server MySQL pada XAMPP belum berjalan atau nilai DB_DATABASE, DB_USERNAME, dan DB_PASSWORD pada .env belum sesuai.

6. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Laravel memudahkan pengelolaan database melalui konfigurasi file .env, migration, model Eloquent, dan seeder. Koneksi database berhasil dibuat dengan mengatur file .env dan menyiapkan database db_laravel. Migration berhasil membentuk tabel mahasiswa beserta kolom-kolomnya secara otomatis, sedangkan model Mahasiswa menghubungkan aplikasi dengan tabel tersebut melalui properti $table dan $fillable. Form input, controller, route, dan view bekerja sama sehingga data mahasiswa dapat disimpan dan ditampilkan kembali dari database. Penggunaan seeder dengan Faker juga berhasil menghasilkan 20 data dummy secara otomatis sehingga pengujian aplikasi menjadi lebih cepat. Secara keseluruhan seluruh tujuan praktikum dapat tercapai dengan baik.

7. Link GitHub

Kode program lengkap praktikum ini dapat diakses pada repositori berikut.

https://github.com/davinaileenskie1-hue/PABW-Semester-3/tree/main/Praktikum_4/PRK_4